

Computational Thinking with Python

Session 5 – Regular Expressions

STUDENTS WILL LEARN TO MATCH AND EXTRACT TEXT PATTERNS

At the end of this session, you should be able to comprehend, trace, adapt and create Python code that:

- Uses the **re module** for pattern matching
- Writes basic patterns with **literals** and **wildcards**
- Applies **quantifiers** (*, +, ?, {n}) to match repetitions
- Uses **character sets** [] and **anchors** (^, \$)
- Extracts data using **capturing groups** ()

AGENDA

- 1 Introduction
- 2 Basic Patterns
- 3 Quantifiers & Sets
- 4 Common Pitfalls

AGENDA

1

Introduction

2

Basic Patterns

3

Quantifiers & Sets

4

Common Pitfalls

REGULAR EXPRESSIONS DESCRIBE PATTERNS IN TEXT

What are Regular Expressions?

- Compact textual representation of string patterns
- **Descriptive** rather than literal
- Universal technology across programming languages

Key insight: Unlike arithmetic expressions that evaluate to values, regular expressions *match* text patterns.

Why Use Them?

- Find patterns in messy data
- Validate input formats (emails, phone numbers)
- Extract information from text
- Clean and transform data

Essential for data cleaning!

PATTERN MATCHING REPLACES COMPLEX CONDITIONAL LOGIC

Problem: Validate email addresses like `someone@au.dk`

Without Regex

```
# Horror show of nested if/else
if "@" in email:
    parts = email.split("@")
    if len(parts) == 2:
        if "." in parts[1]:
            # More checks...
```

With Regex

```
import re

pattern = r"[\w.-]+@[ \w.-]+\.\w+"
if re.match(pattern, email):
    print("Valid!")
```

(Don't worry about the exact symbols yet – we build them up next.)

PYTHON'S RE MODULE PROVIDES PATTERN MATCHING FUNCTIONS

```
import re  
pattern = re.compile(r"abc") # Compile pattern for reuse
```

- `match()` Check if pattern matches at **beginning** of string
- `search()` Find **first location** where pattern matches
- `findall()` Find **all substrings** matching the pattern (returns list)

```
pattern.search("xyzabc") # compiled form  
re.search(r"abc", "xyzabc") # direct form (used from here on)
```

AGENDA

- 1 Introduction
- 2 Basic Patterns**
- 3 Quantifiers & Sets
- 4 Common Pitfalls

LITERAL PATTERNS MATCH EXACT CHARACTER SEQUENCES

Simple Matching

The pattern `r"abc"` matches any string containing `abc`:

✓ `"abc"`, `"abcdef"`, `"xyzabc"`

✗ `"ab c"`, `"ABC"`, `"cba"`

(shown using `re.search`)

Note: Matching is **case-sensitive** by default!

Finding a Literal

```
text = "cat catalog Cat"

re.findall(r"cat", text)
# ['cat', 'cat']

# case-sensitive: "Cat" missed
re.findall(r"Cat", text)
# ['Cat']
```

WILDCARDS MATCH CATEGORIES OF CHARACTERS

Common Wildcards

- . Any character (except newline)
- \d Any **digit** (0-9)
- \D Any non-digit
- \w Any **word character** (a-z, A-Z, 0-9, _)
- \W Any non-word character
- \s Any **whitespace** (space, tab, newline)
- \S Any non-whitespace

Examples

```
text = "Price: $42.99"

re.findall(r"\d", text)
# ['4', '2', '9', '9']

re.findall(r"\w+", text)
# ['Price', '42', '99']
```

+ = one or more; more in Section 3

Use `r"..."` for raw strings!

RAW STRINGS KEEP BACKSLASHES INTACT FOR THE REGEX ENGINE

The Problem

Normal Python strings treat `\` as an **escape character**:

`"\n"` becomes a newline

`"\t"` becomes a tab

`"\b"` becomes a backspace!

So `"\b"` reaches the regex engine as a **backspace** – not the word boundary you meant.

The Fix: Raw Strings

Prefix the string with `r` so Python keeps every backslash **literally**:

```
"\b" # one backspace character
```

```
r"\b" # the two characters \ and b
```

```
# regex now sees \b = word boundary  
re.findall(r"\bcat\b", text)
```

Always write regex patterns as `r"..."`

ANCHORS MATCH POSITIONS, NOT CHARACTERS

Position Anchors

- ^ Start of string
- \$ End of string
- \b Word boundary

Important: Anchors are **zero-width** – they match a position, not text.

Examples

```
text = "Python is fun"

re.search(r"^Python", text)
# Match (starts with Python)

re.search(r"fun$", text)
# Match (ends with fun)

re.findall(r"\bis\b", text)
# ['is'] (whole word only)
```

ESCAPING, ALTERNATION, AND GROUPING HANDLE SPECIAL SYNTAX

Characters to Escape

These characters have special meaning in regex:

`. ^ $ * + ? { } [ ] \ | ( )`

To match them literally, prefix with `\`

```
# Match literal dot
re.search(r"\.", "3.14") # Match

# Match dollar sign
re.search(r"\$", "$99") # Match
```

OR and Grouping

`|` OR (alternative)

`()` Grouping

```
# Match google.de OR google.com
r"google\.(de|com)"
```

```
# Careful with precedence!
r"cat|dog" # cat OR dog
r"(cat|dog)s" # cats OR dogs
```

AGENDA

- 1 Introduction
- 2 Basic Patterns
- 3 Quantifiers & Sets**
- 4 Common Pitfalls

QUANTIFIERS SPECIFY HOW MANY TIMES A PATTERN REPEATS

Basic Quantifiers

*	Zero or more
+	One or more
?	Zero or one (optional)
{n}	Exactly n times
{n,m}	Between n and m times
{n,}	At least n times

Examples

```
# Match phone numbers
r"\d{3}-\d{4}"
# Matches: 123-4567

# Optional area code
r"(\d{3}-)?\d{3}-\d{4}"
# Matches: 123-4567
# Matches: 089-123-4567
```

*** ALLOWS ZERO REPETITIONS, + REQUIRES AT LEAST ONE, ? MAKES IT OPTIONAL**

```
# * means zero or more  
r"ab*c"  
# Matches: ac, abc, abbc, abbbc
```

```
# + means one or more  
r"ab+c"  
# Matches: abc, abbc, abbbc  
# NOT: ac
```

```
# ? means optional  
r"colou?r"  
# Matches: color, colour  
  
# .* matches anything  
r"Hello.*World"  
# Matches: Hello World  
# Matches: Hello Beautiful World
```

CHARACTER SETS DEFINE ALLOWED CHARACTERS AT A POSITION

Character Set Syntax

- `[abc]` Match a, b, or c
- `[a-z]` Match any lowercase letter
- `[A-Z]` Match any uppercase letter
- `[0-9]` Match any digit
- `[^abc]` Match anything *except* a, b, c

Combine ranges: `[a-zA-Z0-9]`

Examples

```
# Match hex color codes
r"#[0-9a-fA-F]{6}"
# Matches: #FF5733, #abc123

# Match Danish postal codes
r"[0-9]{4}"
# Matches: 8000, 1050
```

GROUPS CAPTURE MATCHED SUBSTRINGS FOR EXTRACTION

Capturing Groups

Parentheses () create numbered groups:

- Group 0: Entire match
- Group 1, 2, 3...: Each () from left to right

Access with `match.group(n)`

The same () used for alternation also **capture** what they match.

Example

```
pattern = r"(\w+)@(\w+\.\w+)"
text = "email: john@example.com"

m = re.search(pattern, text)
m.group(0) # john@example.com
m.group(1) # john
m.group(2) # example.com
```

FLAGS MODIFY HOW PATTERNS ARE MATCHED

Flag	Effect	Example
<code>re.I</code>	Case-insensitive matching	<code>re.search(r"python", text, re.I)</code>
<code>re.M</code>	<code>^</code> and <code>\$</code> match line boundaries	Multiline text processing
<code>re.S</code>	<code>.</code> matches newlines too	Match across lines

```
# Case-insensitive search
text = "Python is PYTHON"
re.findall(r"python", text, re.I) # ['Python', 'PYTHON']
```

AGENDA

- 1 Introduction
- 2 Basic Patterns
- 3 Quantifiers & Sets
- 4 Common Pitfalls**

GREEDY MATCHING AND FORGOTTEN RAW STRINGS CAUSE MOST BUGS

Forgetting Raw Strings

```
# Wrong: \b becomes a backspace
re.search("\bcat", text) #
silently wrong

# Correct: raw string
re.search(r"\bcat", text) # \b =
word boundary
```

Always use `r"..."` for regex!

Greedy Matching

```
text = "A=1; B=2; C=3"

# greedy: grabs too much
re.findall(r"=.*;", text)
# ['=1; B=2;']

# lazy: ? stops early
re.findall(r"=.*?;", text)
# ['=1;', '=2;']
```

`.*` is greedy – `?` makes it **lazy**.

REGULAR EXPRESSIONS ENABLE POWERFUL TEXT PATTERN MATCHING

- ✓ **re module** provides `match()`, `search()`, and `findall()`
- ✓ **Wildcards:** `\d` (digits), `\w` (word chars), `\s` (whitespace)
- ✓ **Quantifiers:** `*` (0+), `+` (1+), `?` (0-1), `{n}` (exactly n)
- ✓ **Character sets:** `[abc]` matches a, b, or c; `[^abc]` matches anything except
- ✓ **Groups:** `()` capture matched substrings for extraction

Always use raw strings `r"..."` for regex patterns!

NOW PUT IT ALL TOGETHER WITH HANDS-ON PRACTICE

Practice: Open `01_Regex_Exercises` in your Codespace

Complete: `01_predict.py` - `10_make.py`

Predict → Investigate → Modify → Make

Thank You!

Nikolai Stein

